CISC 886 – Cloud Computing

Lab 8 Assignment: Automating Multi-Container Deployment with CI/CD

School of Computing, Queen's University, Kingston, Canada

Overview

This assignment builds on Lab 8 and Lab 6. You will extend the CI/CD pipeline you built in the lab to deploy a multi-container application: the Node.js app connected to a database, coordinated with Docker Compose. Every time you push code, the pipeline will rebuild the image, push it to Docker Hub, pull the latest files and image onto EC2, and bring the full stack up automatically.

If you completed the Lab 6 exercise, you can reuse the database setup and `docker-compose.yml` you already built. If you prefer to start fresh, any supported database (PostgreSQL, MySQL, or MongoDB) is acceptable.

Task 1 – Add a Database Service to the Repository

In the lab, the app stored its task data in memory. Extend the app to read from and write to a database, and describe both services in a `docker-compose.yml` inside the `lab8-cicd` repository.

1. Add the appropriate database driver to the app and update `app.js` to connect to and query the database. Create an initialisation script that seeds the database with the same sample tasks as before.

2. Create a `docker-compose.yml` at the root of the repository. The file should:

- Build the app image from the local `Dockerfile`
- Expose port `3000`
- Pass the database connection string to the app as an environment variable, using the database service name as the hostname
- Use the appropriate official database image
- Mount the initialisation script so the database is seeded on first start
- Use a named volume for database storage

3. Test the compose stack on EC2 before touching the pipeline:

```
cd ~/lab8-cicd
docker compose up -d
docker compose ps
curl localhost:3000/tasks
```

Write-up 1: State which database you chose. Explain in one or two sentences why the app container uses the database service name as the hostname instead of `localhost` or an IP address.

Submit with Task 1:

- Screenshot of `docker compose ps` showing both services running
- Screenshot of `curl localhost:3000/tasks` returning data from the database
- Your written answer to Write-up 1

Task 2 – Update the Pipeline to Deploy the Compose Stack

The current `deploy.yml` uses individual `docker pull`, `docker stop`, and `docker run` commands in the SSH step. This approach does not work for a multi-container application. Update the workflow so the pipeline deploys the full stack using Docker Compose.

1. On EC2, clone the repository once so the compose file can be fetched on every subsequent pipeline run:

```
git clone https://github.com/<your-github-username>/lab8-cicd.git ~/lab8-cicd
```

> 💡 **Note:** Because the repository is public, cloning and pulling over HTTPS requires no credentials on EC2.

2. Open `deploy.yml` in VS Code and replace the `script` block in the `Deploy to EC2` step with the following:

```
script: |
  cd ~/lab8-cicd
  git pull
  docker compose pull
  docker compose down
  docker compose up -d
```

`git pull` fetches the latest `docker-compose.yml` and any other repository files. `docker compose pull` downloads the latest built image from Docker Hub. `docker compose up -d` starts the full stack.

3. Commit and push:

```
git add .github/workflows/deploy.yml
git commit -m "Update pipeline to deploy with Docker Compose"
git push origin main
```

Go to the **Actions** tab and confirm all steps complete with green checkmarks.

4. Verify on EC2:

```
docker compose ps
curl http://localhost:3000/tasks
```

Write-up 2: Paste the output of `docker compose ps` after the automated deploy. Explain in one sentence how the two containers are able to communicate with each other without exposing the database port publicly.

Submit with Task 2:

- Screenshot of the GitHub Actions run showing all steps green
- Screenshot of `docker compose ps` showing both containers running after the automated deploy
- Screenshot of `curl localhost:3000/tasks` showing data returned from the database
- Your written answer to Write-up 2

Task 3 – Trigger a Full Round-Trip Change

The pipeline is now deploying the full stack automatically. Make a visible change to confirm that a single `git push` is all it takes to update the live application.

1. Open `app.js` in VS Code and add a new task to the tasks array (or to your database seed, depending on your implementation). For example:

```
{ id: 7, name: 'Tea', status: 'pending' }
```

2. Commit and push:

```
git add .
git commit -m "Add Tea to task list"
git push origin main
```

3. Watch the **Actions** tab. Once the run completes, verify the change on EC2:

```
curl http://localhost:3000/tasks
```

The new task should appear in the response without any manual SSH, `docker pull`, or `docker run` commands.

Write-up 3: At what point in the pipeline does Docker Hub act as the intermediary between the GitHub Actions runner and the EC2 instance? Explain in one or two sentences why this matters if the EC2 server cannot directly connect to the GitHub Actions runner.

Submit with Task 3:

- Screenshot of the GitHub Actions run triggered by your commit
- Screenshot of `curl localhost:3000/tasks` showing the new task in the response
- Your written answer to Write-up 3

Cleanup

🚨 **Caution:** EC2 instances accrue charges while running. Always terminate your instance after the assignment.

On the EC2 instance:

```
cd ~/lab8-cicd
docker compose down -v
docker rmi <your-dockerhub-username>/lab6-app:latest
```

In the AWS Console:

- EC2 → Instances → select your instance
- **Instance State** → **Terminate instance** → confirm
- Wait for the state to show **Terminated**, then select it again and choose **Actions** → **Instance Settings** → **Delete** to remove it from the list

⚠️ **Warning:** Terminating an instance is irreversible. Make sure you have submitted your work before terminating.